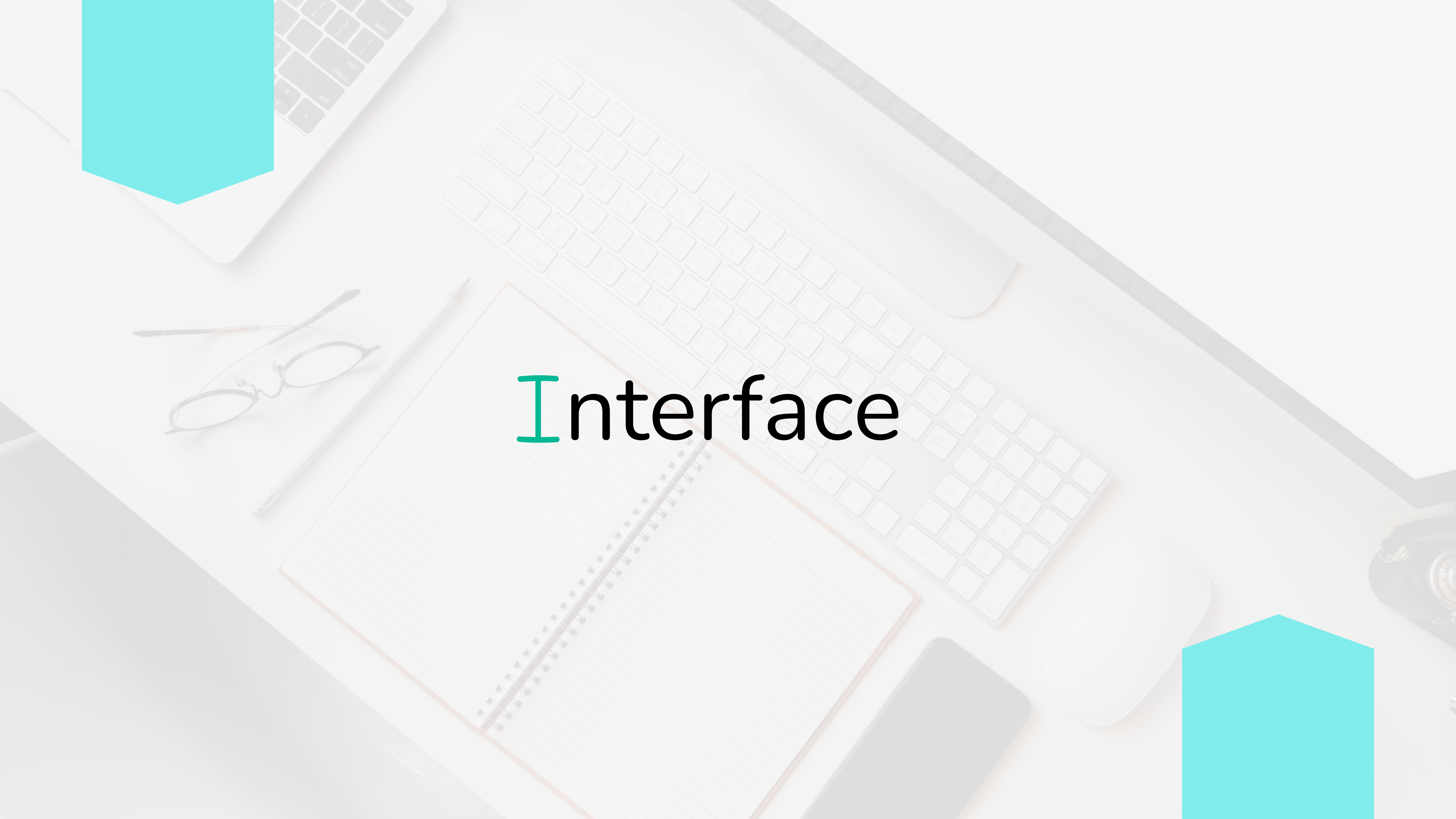


A top-down view of a workspace featuring a laptop, a separate keyboard, a mouse, a spiral notebook, a pair of glasses, and a pen. The word 'Interface' is centered over the image. The letter 'I' is teal, while the rest of the word is black. Two teal geometric shapes, a pentagon in the top-left and a hexagon in the bottom-right, are also present.

Interface

QUE ES UN INTERFACE?

- Una interfaz define un contrato. Cualquier clase que implemente ese contrato debe proporcionar una implementación de lo que se encuentre definido en la interfaz.
- Una interfaz puede definir una implementación predeterminada de métodos.
- Puede declarar una interfaz como declara una clase o estructura, pero usando la palabra clave de interface.
- Dentro de una interfaz, los métodos y propiedades tienen su firma declarada pero no su definición.

IMPLEMENTACIÓN

supongamos que queremos que nuestro jugador interactúe con “X” cantidad de objetos que se encuentra en escena. Para lograr eso, primero tenemos que crear la siguiente interfaz:

```
using UnityEngine;

5 referencias
public interface IInteractable
{
    3 referencias
    void Acccion();
}
```

Nombre: La interfaz se llama Iinteractable. Tenga en cuenta que el nombre comienza con la letra **I**. Esta es una convención para facilitar la identificación de las interfaces; no es obligatorio, pero le recomendamos que siga esta convención.

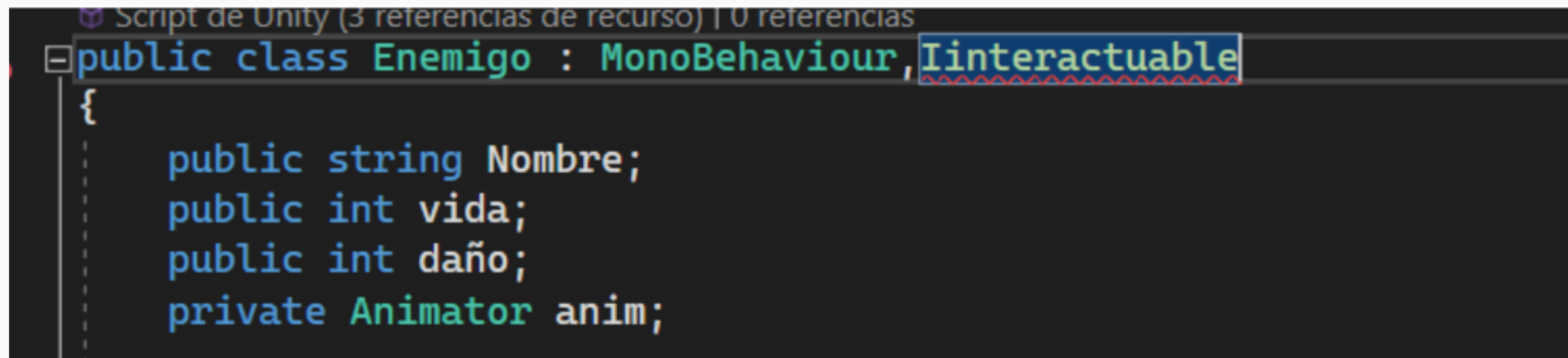
Propiedad: Puedes utilizar cualquier propiedad en una interfaz es una declaración.

La propiedad es del tipo que vos le asignes y debe tener al menos un captador público. La propiedad puede tener opcionalmente un getter o un setter de cualquier nivel de protección.

Método: La segunda línea de la interfaz es la declaración de un solo método llamado. Este método tiene un tipo de retorno de vacío.

Todos los métodos y propiedades en una interfaz son automáticamente públicos y, por lo tanto, al declararlos, no se requiere la palabra clave pública. Además, dado que las definiciones de métodos y propiedades dependen de las clases y estructuras de implementación, no se requieren cuerpos.

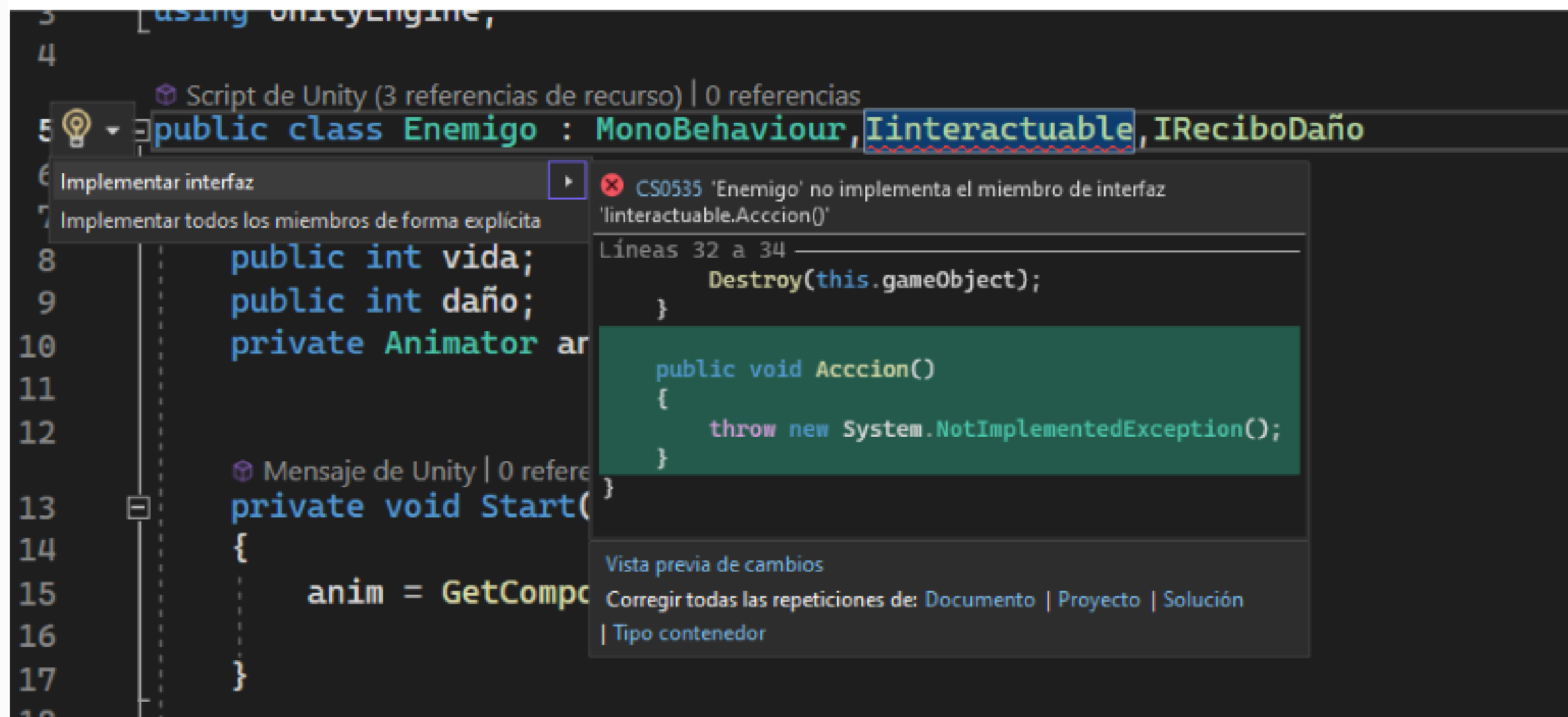
Una vez declarada nuestra interfaz, se lo vamos aplicar a los objetos que queramos que sean interactuados por el jugador:



```
Script de Unity (3 referencias de recurso) | 0 referencias  
public class Enemy : MonoBehaviour, Interactable  
{  
    public string Nombre;  
    public int vida;  
    public int daño;  
    private Animator anim;
```

Se puede agregar después de los dos puntos (:) como si fuera una herencia o simplemente agregándole después de una coma..

Como vemos en pantalla, el compilador nos está avisando que algo está mal, eso es porque todavía no lo estamos implementando como corresponde.



```
3 using UnityEngine;
4
5 public class Enemigo : MonoBehaviour, IInteractable, IReciboDaño
6 {
7     public int vida;
8     public int daño;
9     private Animator anim;
10
11     private void Start()
12     {
13         anim = GetComponent<Animator>();
14     }
15
16     public void Acccion()
17     {
18         Destroy(this.gameObject);
19     }
20 }
```

Script de Unity (3 referencias de recurso) | 0 referencias

Implementar interfaz

Implementar todos los miembros de forma explícita

CS0535 'Enemigo' no implementa el miembro de interfaz 'IInteractable.Acccion()'

Líneas 32 a 34

```
Destroy(this.gameObject);
}

public void Acccion()
{
    throw new NotImplementedException();
}
```

Mensaje de Unity | 0 referencias

private void Start()

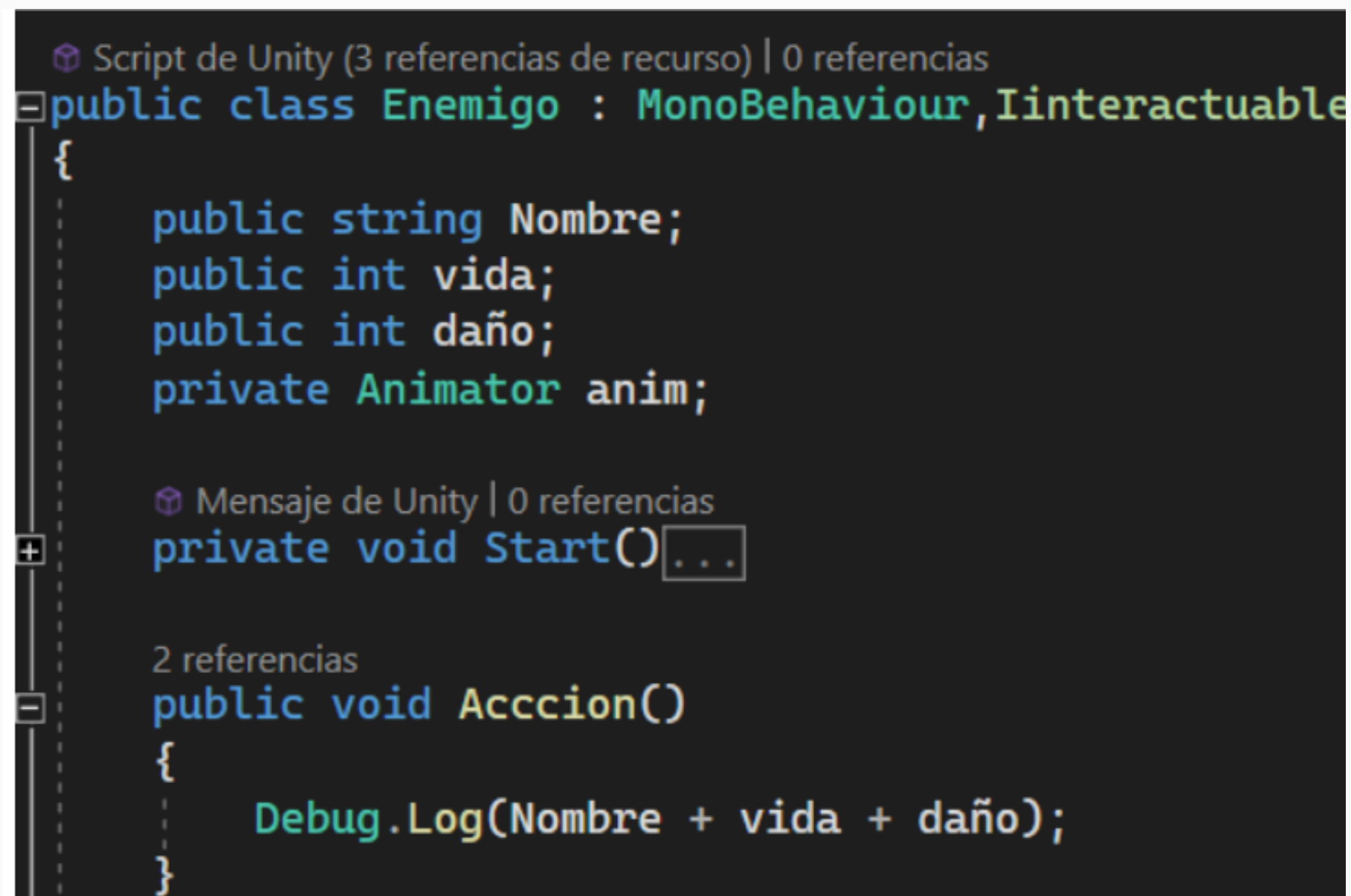
anim = GetComponent<Animator>();

Vista previa de cambios

Corregir todas las repeticiones de: Documento | Proyecto | Solución

| Tipo contenedor

Una vez que utilicemos los mismos parámetros y métodos de la interfaz en nuestro código, ya estará implementado correctamente. Ahora solo queda agregar la lógica que queramos para nuestro juego.



```
Script de Unity (3 referencias de recurso) | 0 referencias
public class Enemy : MonoBehaviour, Interactable
{
    public string Nombre;
    public int vida;
    public int daño;
    private Animator anim;

    Mensaje de Unity | 0 referencias
    private void Start()...

    2 referencias
    public void Acccion()
    {
        Debug.Log(Nombre + vida + daño);
    }
}
```

En este caso, la utilizaré para mostrar el nombre, vida y daño del enemigo en pantalla.

¡Excelente! Ya tenemos implementada nuestra Interfaz, el siguiente paso es llamarla desde otro script. Para eso, voy a utilizar el script del jugador para que lo active cuando esté cerca del objeto deseado.

```
1 referencia
private void InteractuarConObjeto()
{
    RaycastHit2D hit = Physics2D.Raycast(transform.position, Vector2.up, distanciaRayCast);

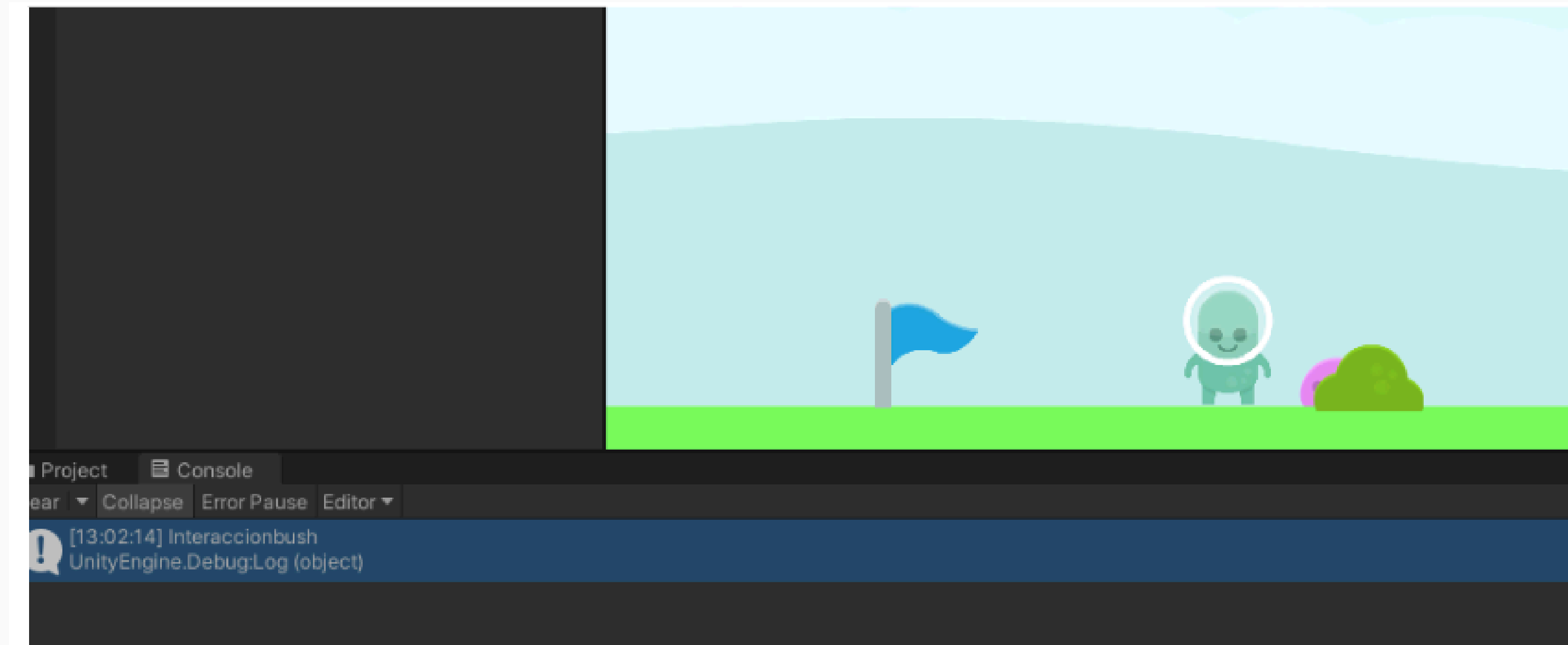
    Debug.DrawRay(transform.position, Vector2.up * distanciaRayCast, Color.red);

    if (hit.collider.GetComponent<IInteractable>() != null)
    {
        Debug.Log("Interaccion" + hit.collider.name);

        hit.collider.GetComponent<IInteractable>().Accion();
    }
    else
        Debug.Log("No estoy interactuando con objetos");
}
```

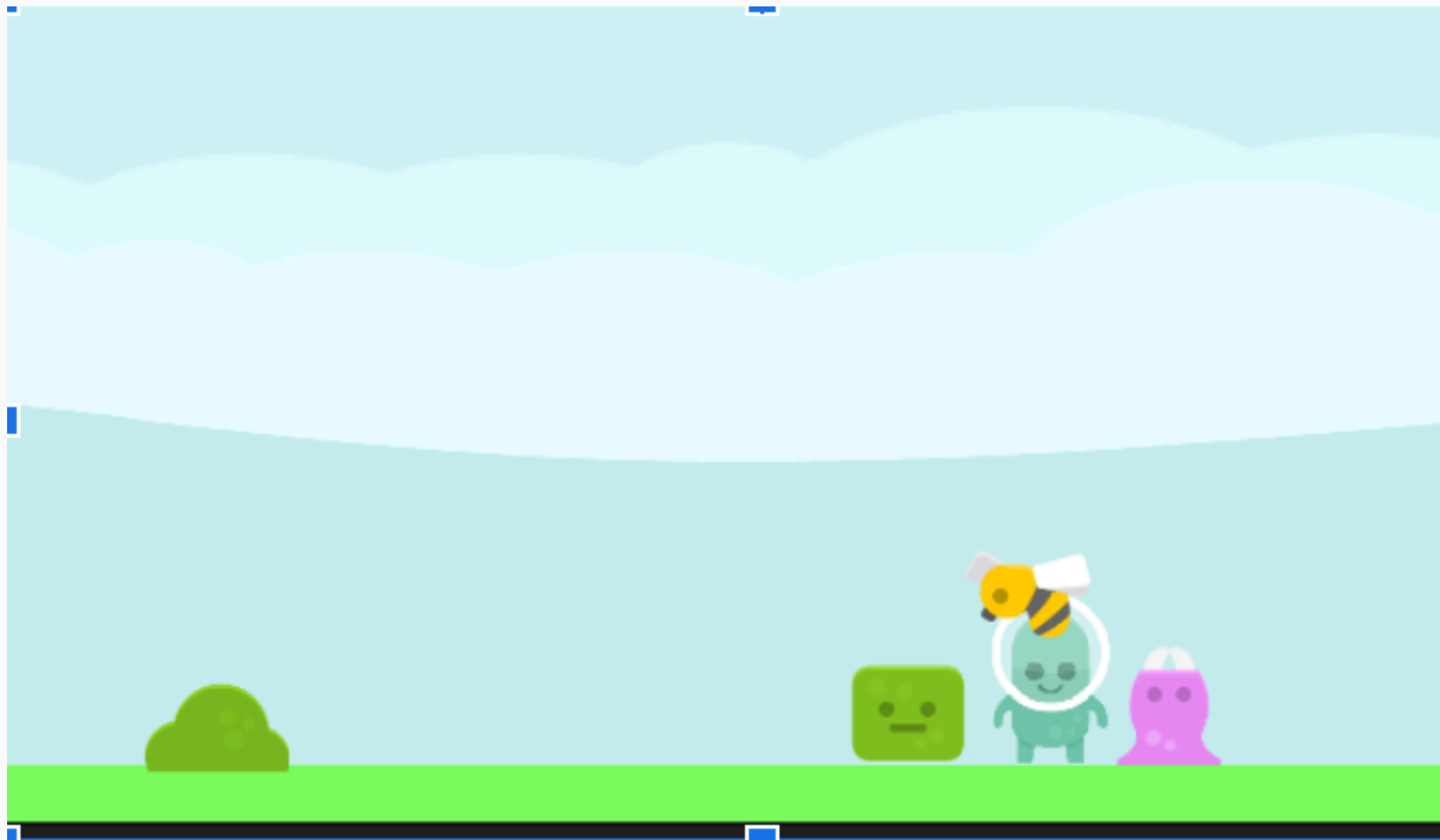
Simplemente detectamos al enemigo a través de un Raycast y comprobamos si tiene la interfaz linteractable, si es correcto, le decimos que llame al método Accion();

Recordemos que el método **Accion()**; solo va mostrar información en pantalla, pero realmente pueden hacer la lógica que necesitamos en nuestro juego.



OTRO CASO DE USO

Supongamos que ahora nuestro Personaje quiere eliminar a distintos enemigos en pantalla.



Primero tenemos que crear la interfaz adecuada:

```
4
    3 referencias
5  public interface IReciboDaño
6  {
7      2 referencias
      void RecibirDaño(int dmg);
8  }
9
```

Creamos una Interfaz llamada **IReciboDaño** donde va tener un método llamado Recibir Daño que devuelve vacío y requiere un parámetro de tipo entero.

Implementamos la interfaz en el script del enemigo junto con su lógica.

```
Script de Unity (3 referencias de recurso) | 0 referencias
public class Enemigo : MonoBehaviour, Interactable, IReciboDaño
{
    public string Nombre;
    public int vida;
    public int daño;
    private Animator anim;

    Mensaje de Unity | 0 referencias
    private void Start()...

    2 referencias
    public void Acccion()...

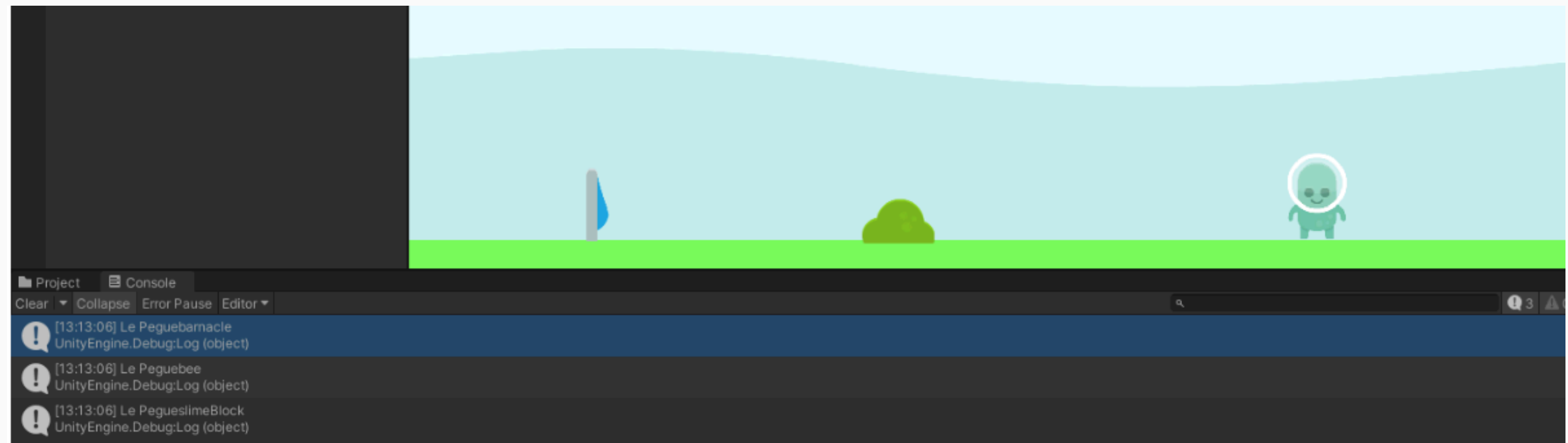
    2 referencias
    public void RecibirDaño(int dmg)
    {
        vida -= dmg;
        if (vida <= 0)
        {
            anim.Play("Muerte");
        }
    }
}
```

Ahora solo nos queda llamar a su método desde el script del jugador:

```
1 referencia
void Atacar()
{
    Collider2D[] hitColliders = Physics2D.OverlapCircleAll(transform.position, distanciaOverlap);

    foreach (var hitCollider in hitColliders)
    {
        if (hitCollider.GetComponent<IReciboDaño>() != null)
        {
            Debug.Log("Le Pegue" + hitCollider.name);
            hitCollider.GetComponent<IReciboDaño>().RecibirDaño(Daño);
        }
    }
}
```

Y Guala! ya tenemos a nuestro aniquilador de monstruos.





CONCLUSIÓN

